

AMERICAN LATIN CLASS ATTENDANCE SYSTEM

Business Rules Api Test Report

Table of Contents

- Business Rules API Test Report
 - Scope
 - Counts
 - RDS Demo Seed Evidence
 - Formal Business Rules From Requirements
 - API-Verifiable Rule Matrix
 - R-01 Public Enrollment Request Queue
 - R-02 Google OAuth Login
 - R-03 Application-Owned Roles
 - R-04 Secure Session Lifecycle
 - R-05 No-Store Private Cache Rule
 - R-06 Role-Based Authorization
 - R-07 Academic Catalog Management
 - R-08 Dance Category And Style Management
 - R-09 Student Attendance Uniqueness
 - R-10 Teacher Check-In And Check-Out
 - R-11 Absence Justification Workflow
 - R-12 Scholarship Candidate Rule
 - R-13 Scholarship Evaluation Rule
 - R-14 Level Promotion Candidate Rule
 - R-15 Level Promotion Evaluation Rule
 - R-16 Reports, Teacher Payment And Audit
 - Postman Verification Guidance
 - Automated Validation Evidence

Business Rules API Test Report

Generated: 2026-06-24

Scope

This report maps the American Latin Class business rules to API URIs and manual/automated test methods. It also records the demo data loaded into the AWS RDS PostgreSQL database for verification.

Counts

- Formal business rules declared in `02Requirements/requirements.md` : 10.
- API-verifiable business rule areas implemented by the system: 16.
- API endpoints by `METHOD + URI` : 53.
- Unique API URI patterns: 35.
- Postman collection requests available for manual validation: 63.
- RDS demo records inserted in this run: 61.

RDS Demo Seed Evidence

The seed was executed from the Core Business API EC2 instance against the configured RDS PostgreSQL database using Prisma ORM.

- Seed prefix: `BRDEMO-20260624132922`
- Inserted records: 61
- Script: `06Code/scripts/seed-business-rule-demo-data.js`
- Executed command on EC2: `node scripts/seed-business-rule-demo-data.js`
- Reusable local command added to the project: `npm run db:seed:business-rules`

Table area	Records inserted
Enrollment requests	8
Students	6
Teachers	3
Class groups	3
Class sessions	10
Student attendance records	19
Teacher attendance records	3
Absence justifications	2
Scholarship evaluations	2
Level promotion evaluations	2

Table area	Records inserted
Audit logs	3
Total	61

RDS table counts after the seed:

Table area	Total rows after seed
Enrollment requests	9
Users	2
Students	6
Teachers	3
Class groups	3
Class sessions	10
Student attendance records	19
Teacher attendance records	3
Absence justifications	2
Scholarship evaluations	2
Level promotion evaluations	2
Audit logs	3

Formal Business Rules From Requirements

ID	Formal business rule
BR-01	Scholarship candidate requires at least 90% attendance in a two-month period.
BR-02	Scholarship approval is never automatic; directors must register theory and practice evaluation results.
BR-03	Scholarship percentages allowed are 25%, 50%, 75% and 100%.
BR-04	Level promotion applies to B1 students moving to B2.
BR-05	Level promotion approval requires attendance evidence, consistency score, theory score and practice score.

ID	Formal business rule
BR-06	A student can have only one attendance record per class session.
BR-07	A teacher can have only one open check-in at a time.
BR-08	Roles are owned by the application and are not inferred from Google claims.
BR-09	Private pages and APIs must use <code>Cache-Control: no-store</code> .
BR-10	Administrative and academic state-changing actions must be audited.

API-Verifiable Rule Matrix

R-01 Public Enrollment Request Queue

What it does: visitors can submit enrollment requests, and directors/admin users can review the request queue.

URIs:

- `POST /api/v1/enrollment-requests`
- `GET /api/v1/enrollment-requests`

Methods to test:

- `POST` valid name/email/body -> expect `201`, status defaults to `pending`.
- `POST` invalid email or short name -> expect `422`.
- `GET` anonymous -> expect `401`.
- `GET` Student/Teacher -> expect `403`.
- `GET` BranchDirector/GeneralDirector/Admin -> expect `200`.

R-02 Google OAuth Login

What it does: the backend exposes the Google client ID, validates Google ID tokens, links users by `google_sub`, and issues an application session cookie.

URIs:

- `GET /api/v1/auth/config`
- `POST /api/v1/auth/google`

Methods to test:

- `GET` config -> expect `200` and the configured Google client ID.
- `POST` valid Google ID token -> expect `200` and `alc_session` cookie.

- `POST` malformed/invalid token -> expect `401`.
- `POST` with token audience mismatch -> expect `401`.

R-03 Application-Owned Roles

What it does: Google is used only for identity; application roles are stored internally and are not inferred from Google profile data.

URIs:

- `POST /api/v1/auth/google`
- `GET /api/v1/users`
- `PATCH /api/v1/users/:id/role`
- `GET /api/v1/roles`
- `GET /api/v1/permissions`

Methods to test:

- `POST` first Google login -> expect new user role `Student`.
- `PATCH` role as Admin -> expect `200`.
- `PATCH` role as non-Admin -> expect `403`.
- `PATCH` invalid role -> expect `422`.
- `GET` roles as authenticated user -> expect `200`.
- `GET` users/permissions as Student -> expect `403`.

R-04 Secure Session Lifecycle

What it does: private APIs require a valid session; logout revokes the server-side session and private pages redirect when anonymous or revoked.

URIs:

- `GET /api/v1/auth/me`
- `POST /api/v1/auth/logout`
- `GET /private/dashboard.html`

Methods to test:

- `GET /auth/me` with active session -> expect `200`.
- `GET /auth/me` anonymous -> expect `401`.
- `POST /auth/logout` with active session -> expect `200`.
- `GET /auth/me` after logout -> expect `401`.
- `GET /private/dashboard.html` anonymous -> expect `302` to `/login.html?session=expired`.
- Browser back button after logout -> should not reopen private data.

R-05 No-Store Private Cache Rule

What it does: private pages and APIs must not be cached by the browser or shared proxies.

URIs:

- Protected `/api/v1/*` endpoints
- `GET /private/dashboard.html`
- Private static assets under `/private`

Methods to test:

- `GET` protected API with session -> expect `Cache-Control: no-store`.
- `GET /private/dashboard.html` with session -> expect `Cache-Control: no-store`.
- `GET /private/dashboard.html` after logout/back navigation -> expect redirect instead of cached private content.

R-06 Role-Based Authorization

What it does: protected endpoints enforce Student, Teacher, BranchDirector, GeneralDirector and Admin permissions through middleware.

URIs:

- All protected `/api/v1/*` endpoints except public auth config/login/enrollment submission.

Methods to test:

- Anonymous request to a protected endpoint -> expect `401`.
- Authenticated user with wrong role -> expect `403`.
- Authenticated user with allowed role -> expect the endpoint-specific `200`, `201` or `204` behavior.
- Validate at least one endpoint for each role: Student, Teacher, BranchDirector, GeneralDirector, Admin.

R-07 Academic Catalog Management

What it does: directors/admin users manage branches, students, teachers, class groups and class sessions; authenticated users can read catalog data.

URIs:

- `/api/v1/branches`
- `/api/v1/branches/:id`
- `/api/v1/students`
- `/api/v1/students/:id`
- `/api/v1/teachers`
- `/api/v1/teachers/:id`

- `/api/v1/class-groups`
- `/api/v1/class-groups/:id`
- `/api/v1/class-sessions`
- `/api/v1/class-sessions/:id`

Methods to test:

- `GET` list as authenticated user -> expect `200`.
- `GET` by existing id -> expect `200`.
- `GET` by missing id -> expect `404`.
- `POST` as BranchDirector/GeneralDirector/Admin -> expect `201`.
- `POST` as Student/Teacher -> expect `403`.
- `POST` invalid body -> expect `422`.
- `PATCH` existing id as director/admin -> expect `200`.
- `PATCH` missing id -> expect `404`.

R-08 Dance Category And Style Management

What it does: the system stores and manages dance categories and styles used by class groups.

URIs:

- `/api/v1/dance-categories`
- `/api/v1/dance-categories/:id`
- `/api/v1/dance-styles`
- `/api/v1/dance-styles/:id`

Methods to test:

- `GET` list as authenticated user -> expect `200`.
- `POST` category/style as director/admin -> expect `201`.
- `POST` duplicate unique name -> expect database/application error handling.
- `POST` invalid category/style payload -> expect `422`.
- `PATCH` existing category/style -> expect `200`.
- `PATCH` missing id -> expect `404`.

R-09 Student Attendance Uniqueness

What it does: each student can have only one attendance record per class session.

URIs:

- `POST /api/v1/student-attendance`

Methods to test:

- `POST` valid `studentId`, `classSessionId`, status -> expect `201`.

- **POST** duplicate student/session pair -> expect **409** .
- **POST** missing student id -> expect **404** .
- **POST** missing class session id -> expect **404** .
- **POST** invalid status -> expect **422** .
- **POST** as Student -> expect **403** .
- **POST** as Teacher/BranchDirector/GeneralDirector/Admin -> expect **201** when data is valid.

R-10 Teacher Check-In And Check-Out

What it does: a teacher can have only one open check-in at a time; check-out closes that record for hour calculation.

URIs:

- **POST** `/api/v1/teacher-attendance/check-in`
- **PATCH** `/api/v1/teacher-attendance/:id/check-out`

Methods to test:

- **POST** valid teacher -> expect **201** .
- **POST** same teacher while a check-in is open -> expect **409** .
- **POST** missing teacher -> expect **404** .
- **POST** with missing optional class session -> expect **404** .
- **PATCH** open attendance record -> expect **200** and **checkOutAt** .
- **PATCH** same record twice -> expect **409** .
- **POST/PATCH** as Student -> expect **403** .

R-11 Absence Justification Workflow

What it does: authorized users can justify an attendance record, and directors/admin users approve or reject the justification.

URIs:

- **GET** `/api/v1/absence-justifications`
- **POST** `/api/v1/absence-justifications`
- **PATCH** `/api/v1/absence-justifications/:id/review`

Methods to test:

- **POST** valid attendance record and reason -> expect **201** , status **pending** .
- **POST** missing attendance record -> expect **404** .
- **POST** short reason or invalid evidence URL -> expect **422** .
- **GET** as director/admin -> expect **200** .
- **GET** as Student/Teacher -> expect **403** .
- **PATCH** review as director/admin with **approved** or **rejected** -> expect **200** .
- **PATCH** invalid review status -> expect **422** .

R-12 Scholarship Candidate Rule

What it does: a student is a scholarship candidate when the attendance rate is at least 90% in the evaluated two-month period.

URIs:

- `GET /api/v1/reports/scholarships/:studentId/candidate`

Methods to test:

- `GET` with candidate student -> expect `200`, `candidate: true`, `attendanceRate >= 0.9`.
- `GET` with non-candidate student -> expect `200`, `candidate: false`.
- `GET` with `from` and `to` query dates -> expect the period filter to affect the attendance rate.
- `GET` as Student/Teacher -> expect `403`.
- `GET` as BranchDirector/GeneralDirector/Admin -> expect `200`.

R-13 Scholarship Evaluation Rule

What it does: scholarship approval is a director/admin decision and requires candidate evidence plus theory/practice scores. Percentages are limited to 25, 50, 75 and 100.

URIs:

- `GET /api/v1/scholarship-evaluations`
- `POST /api/v1/scholarship-evaluations`
- `GET /api/v1/reports/scholarships/:studentId/candidate`

Methods to test:

- `GET` evaluations as director/admin -> expect `200`.
- `POST` approved evaluation for candidate with scores ≥ 70 and allowed percentage -> expect `201`.
- `POST` approved evaluation for non-candidate -> expect `422`.
- `POST` invalid percentage such as 30 -> expect `422`.
- `POST` theory/practice outside 0-100 -> expect `422`.
- `POST` as Student/Teacher -> expect `403`.

R-14 Level Promotion Candidate Rule

What it does: level promotion applies to B1 students moving to B2 and uses attendance evidence before director/admin evaluation.

URIs:

- `GET /api/v1/reports/level-promotions/:studentId/candidate`

Methods to test:

- `GET` B1 student with attendance rate $\geq 85\%$ -> expect `candidate: true`.
- `GET` B1 student below 85% -> expect `candidate: false`.
- `GET` non-B1 student -> expect `candidate: false`.
- `GET` with `from` and `to` query dates -> expect the period filter to affect the attendance rate.
- `GET` as Student/Teacher -> expect `403`.

R-15 Level Promotion Evaluation Rule

What it does: directors/admin users register consistency, theory and practice scores; approved candidates move from B1 to B2.

URIs:

- `GET /api/v1/level-promotion-evaluations`
- `POST /api/v1/level-promotion-evaluations`
- `GET /api/v1/reports/level-promotions/:studentId/candidate`
- `PATCH /api/v1/students/:id`

Methods to test:

- `GET` evaluations as director/admin -> expect `200`.
- `POST` approved evaluation for B1 candidate with scores ≥ 70 -> expect `201` and student level updated to `B2`.
- `POST` approved evaluation for non-candidate -> expect `422`.
- `POST` consistency/theory/practice outside 0-100 -> expect `422`.
- `POST` as Student/Teacher -> expect `403`.

R-16 Reports, Teacher Payment And Audit

What it does: consolidated reports are restricted, completed teacher attendance records generate payment totals, and state-changing actions are audited.

URIs:

- `GET /api/v1/reports/branches/summary`
- `GET /api/v1/reports/teachers/:teacherId/payment`
- `GET /api/v1/audit-logs`

Methods to test:

- `GET` branch summary as GeneralDirector/Admin -> expect `200`.
- `GET` branch summary as Student/Teacher/BranchDirector -> expect `403` for consolidated report access.
- `GET` teacher payment with closed check-ins -> expect hours, hourly rate and amount.
- `GET` teacher payment with only open check-ins -> expect open records not counted.

- `GET` audit logs as GeneralDirector/Admin -> expect `200`.
- `GET` audit logs as Student/Teacher/BranchDirector -> expect `403`.

Postman Verification Guidance

Use the files under `postman/` :

- `postman/American-Latin-Class-API.postman_collection.json`
- `postman/American-Latin-Class.postman_environment.json`

Recommended manual order:

1. Select the `American Latin Class - AWS` environment.
2. Run `Auth & Session / Auth Config`.
3. Login through the browser at `https://18-217-255-109.sslip.io/login.html` and obtain a valid application session, or use a real Google ID token for the Postman login request.
4. Run the folders in this order: Auth & Session, Public Enrollment, Identity And RBAC, Catalog CRUD, Attendance And Absences, Reports And Evaluations, Session Teardown.
5. Use the seeded `BRDEMO-20260624132922` records when manual tests need existing attendance, scholarship, promotion, payment or audit data.

Automated Validation Evidence

The API validation suite covers auth, sessions, RBAC, CRUD, attendance, absence, reports, evaluations and private page protection.

- Command: `npm run test:api:validation`
- Evidence file: `03Documentation/api-validation-report.md`
- Result: 95 total cases, 95 passed, 0 failed.

The Jest suite covers integration, auth, RBAC and unit-level rules tests.

- Command: `npm test`
- Result: 4 test suites passed, 15 tests passed, 0 failed.